



## Ottimizzazione 2 - *Architetture Moderne Single-Core*

Principi della Programmazione Parallela  
CdL Informatica - A.A. 2025/2026



Università  
di Catania



**INAF**  
ISTITUTO NAZIONALE  
DI ASTROFISICA

Questa lezione presenta i tratti fondamentali dell'architettura moderna **single-core**, nei loro sviluppi storici e nelle loro radici (presupponendo che la prospettiva storica aiuti la comprensione).

Lo scopo è mostrarvi perché questa conoscenza sia importante per scrivere codice efficiente..

# Riassumendo

**Le CPU diventano più veloci della memoria**

L'accesso alla DRAM centrale diventa un grave collo di bottiglia ("data starvation"). Viene introdotta una **gerarchia di memoria**, che introduce il concetto chiave di località dei dati.

**Le CPU diventano superscalari e acquisiscono capacità out-of-order**

Un core moderno ha più porte e può eseguire più di un'operazione nello stesso ciclo di clock (**Parallelismo a Livello di Istruzione** - ILP).

**Le operazioni sono suddivise in stadi più piccoli e pipelinate**

La produttività (throughput) attesa è maggiore, a condizione che le **pipeline** non si blocchino (a causa di rischi di dati e di controllo).

**I predittori di salto (branch predictors) sono una parte importante del front-end:**

Gestire i salti / **branch** è un fattore chiave per aumentare le prestazioni del codice, prevenendo i blocchi della pipeline.

**Le CPU acquisiscono capacità vettoriali**

Registri speciali eseguono la stessa operazione su più dati (SIMD - **Single Instruction Multiple Data**). Questo è il **Parallelismo a Livello di Dati**.



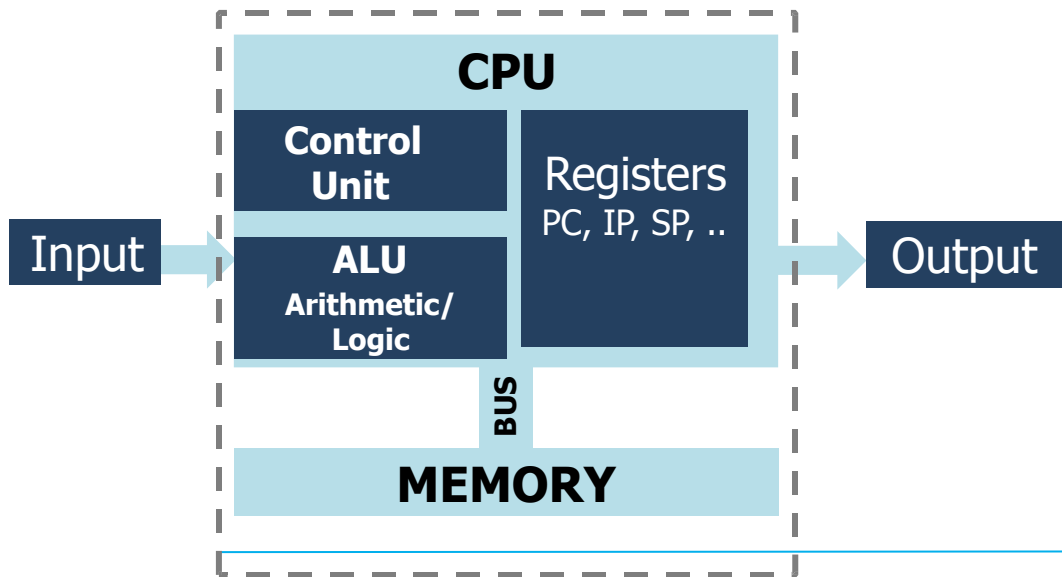
# C'era una volta..



... L'architettura del computer era molto più semplice di oggi.

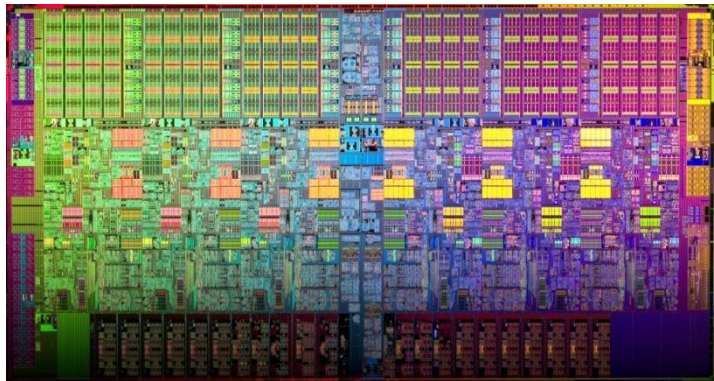
Nell'architettura di Von Neumann:

- C'è solo 1 unità di elaborazione.
- 1 istruzione viene eseguita alla volta.
- La memoria è "piatta":
  - l'accesso a qualsiasi posizione ha sempre lo stesso costo, e
  - l'accesso alla memoria ha lo stesso costo dell'esecuzione di un'operazione.





# Mentre oggi...



**NON è un esempio estremo: Intel Xeon e5600 a 6 core**

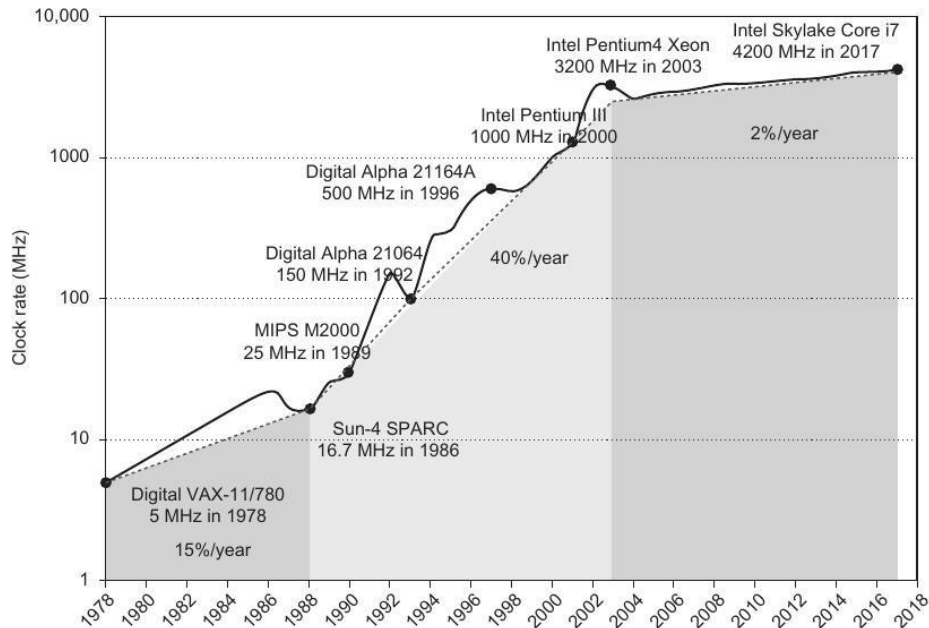
(\*) "costo" in termine di cicli di CPU

- Ci sono **molte** unità di elaborazione.
- Molte istruzioni alla volta possono essere eseguite
- Molti dati possono essere elaborati contemporaneamente.
- Le "istruzioni" sono suddivise in molte micro-operazioni ( $\mu$ ops) più semplici che sono pipelinate.
  - Istruzioni diverse potrebbero avere costi (in termini di cicli di CPU) molto diversi.
- La memoria non è affatto "piatta":
  - esiste una forte gerarchia di memoria
  - l'accesso alla memoria può avere costi diversi a seconda della locazione
  - l'accesso alla RAM è molto più costoso dell'esecuzione di una  $\mu$ op o della lettura di un registro interno.





# L'energia e la potenza sono un problema di progettazione importante



Abbiamo visto che la **potenza** richiesta per transistor è  $C \times V^2 \times f$ . All'incirca la capacità e la tensione del transistor diminuiscono con le dimensioni caratteristiche, mentre il ridimensionamento è molto più complicato per i fili.

Una CPU tipica è passata da  $\sim 2W$  di potenza a  $\sim 100W$ , che è al limite della capacità di raffreddamento ad aria.

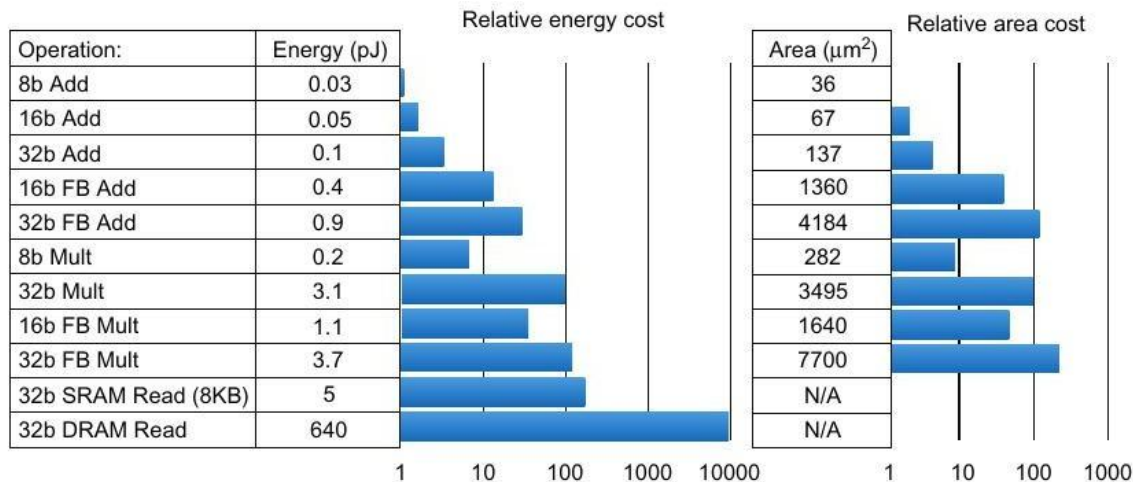


## L'energia e la potenza sono un problema di progettazione importante

Strategie primarie per affrontare il power wall:

1. Spegner i circuiti inattivi (dark silicon).
2. Ridimensionare Tensione e Frequenza sia per i core che per la DRAM.
3. Progettazione della Potenza Termica, o progettazione per il caso tipico.
4. Overclocking per un breve periodo di tempo e possibilmente solo per una frazione del chip.

La potenza statica, che aumenta con il numero di transistor, è un fattore importante per il consumo di potenza e può ammontare a ~25% del consumo totale di potenza.



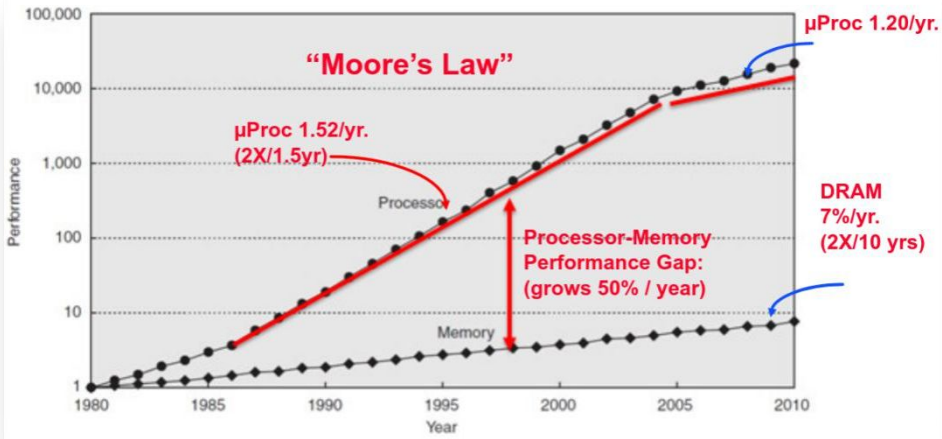
Energy numbers are from Mark Horowitz \*Computing's Energy problem (and what we can do about it)\*. ISSCC 2014  
Area numbers are from synthesized result using Design compiler under TSMC 45nm tech node. FP units used DesignWare Library.

From Hernessey & Patterson, Computer Architecture - A quantitative approach, 6th Ed, 2017



# Inizio anni 90:

## le CPU diventano più veloci della memoria



| Processor               | Alpha 21164                       |                                        |
|-------------------------|-----------------------------------|----------------------------------------|
| Machine                 | AlphaServer 8200                  |                                        |
| Clock Rate              | 300 MHz                           |                                        |
| Memory Performance      | Latency                           | Bandwidth                              |
| I Cache (8KB on chip)   | 6.7 ns (2 clocks)                 | 4800 MB/sec                            |
| D Cache (8KB on chip)   | 6.7 ns (2 clocks)                 | 4800 MB/sec                            |
| L2 Cache (96KB on chip) | 20 ns (6 clocks)                  | 4800 MB/sec                            |
| L3 Cache (4MB off chip) | 26 ns (8 clocks)                  | 960 MB/sec                             |
| Main Memory Subsystem   | 253 ns (76 clocks)                | 1200 MB/sec                            |
| Single DRAM component   | $\approx 60\text{ns}$ (18 clocks) | $\approx 30\text{--}100\text{ MB/sec}$ |

*Preso da un paper del 1997*

La CPU può trascorrere più tempo ad aspettare i dati dalla RAM che ad eseguire operazioni. Questo fa parte del cosiddetto "memory wall".

Qual è la soluzione?





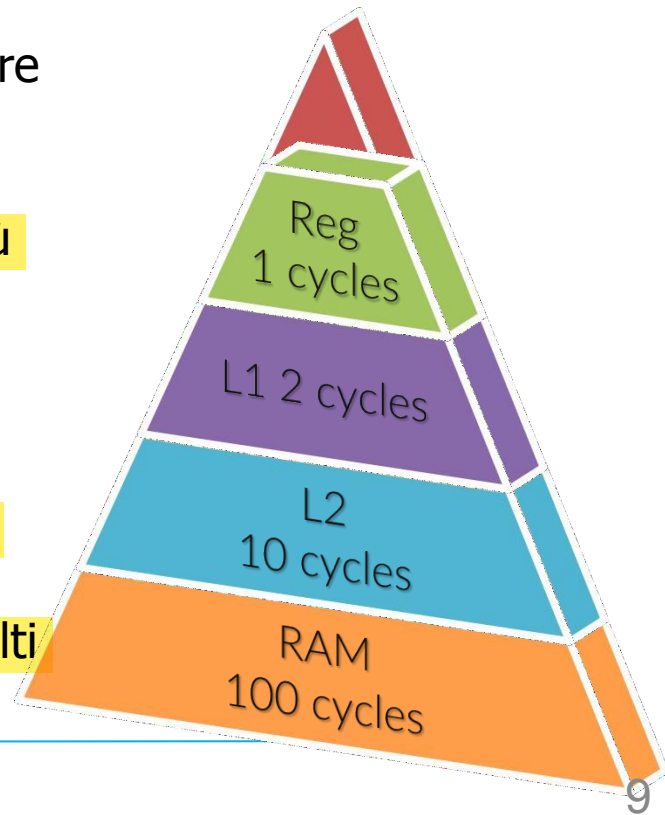
# The cache memory

La soluzione è dotare la CPU di una memoria più veloce, più costosa e più energivora, che deve essere estremamente vicina.

La nuova memoria, chiamata **cache**, sarà molto più piccola della RAM.

La cache stessa ha una gerarchia:

- Livello I: all'interno di ciascun core.
- Livello II: all'interno del core, o condivisa da più core.
- Livello III: all'interno della CPU, condivisa da molti core.





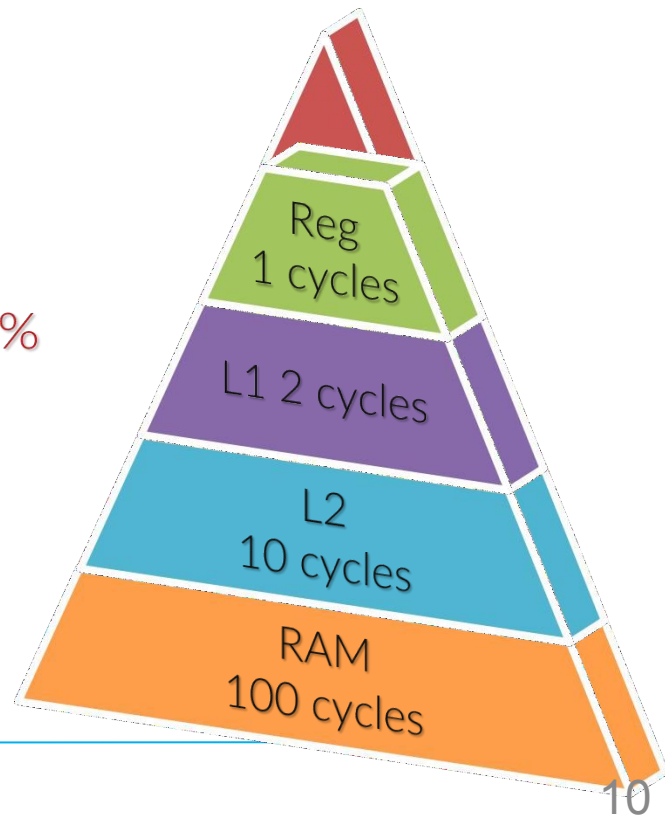
# The cache memory

L1 cache + RAM

$$L_{1cost} + Miss_1 \times RAM_{cost}$$

- 100% L1 hit → 2 cycles
- 99% L1 hit → 3 cycles
- 97% L1 hit → 5 cycles

} 50% to 150%  
slower





# The cache memory

L1 cache + RAM

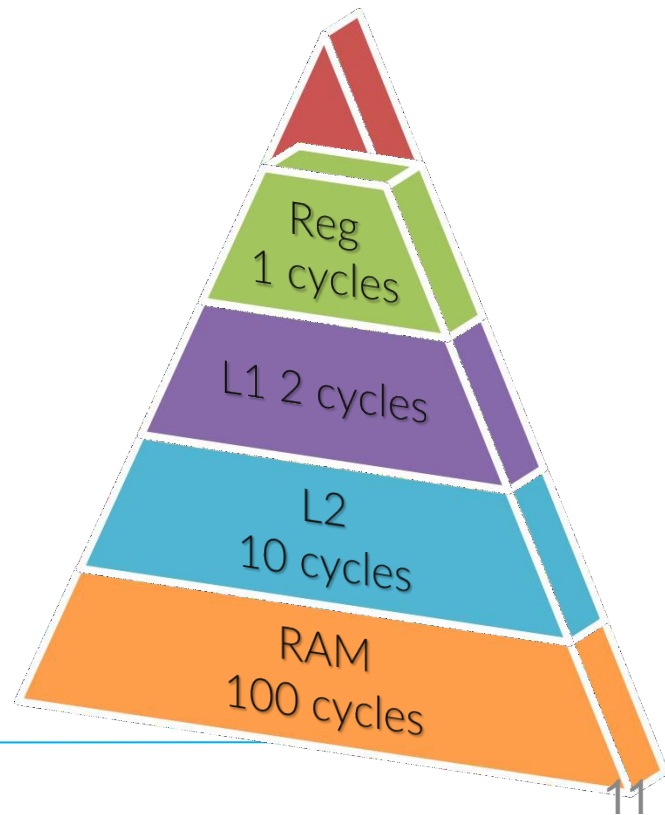
$$L_{1cost} + Miss_1 \times RAM_{cost}$$

- 100% L1 hit → 2 cycles
- 99% L1 hit → 3 cycles
- 97% L1 hit → 5 cycles

L1 cache + L2 cache + RAM

$$L_{1cost} + Miss_1 \times (L2_{cost} + Miss_2 \times RAM_{cost})$$

- 100% L1 hit → 2 cycles
- 99% L1 hit, 100% L2 hit → 2.1cycles
- 97% L1 hit, 100% L2 hit → 2.3 cycles
- 90% L1 hit, 97% L2 hit → 3.3 cycles





# Il principio di località

---

I dati sono definiti "locali" quando risiedono in una piccola porzione dello spazio degli indirizzi acceduta in un breve periodo di tempo

→ è probabile che i dati locali siano nella cache

- **Località temporale:** se un indirizzo viene referenziato, è probabile che venga referenziato presto di nuovo.
- **Località spaziale:** se un indirizzo viene referenziato, è probabile che indirizzi vicini vengano presto referenziati.



# Riepilogo della Cache

**Le 3C  
per i nemici**

## **Miss Compulsory (Obbligatori):**

Inevitabili al primo accesso ai dati.

↳ la cache è vuota all'inizio

## **Miss di Capacità:**

Spazio insufficiente nella cache.

## **Miss di Conflitto:**

Svuotamento della cache (cache trashing) dovuto alla mappatura dei dati sulle stesse linee di cache.





# Riepilogo della Cache

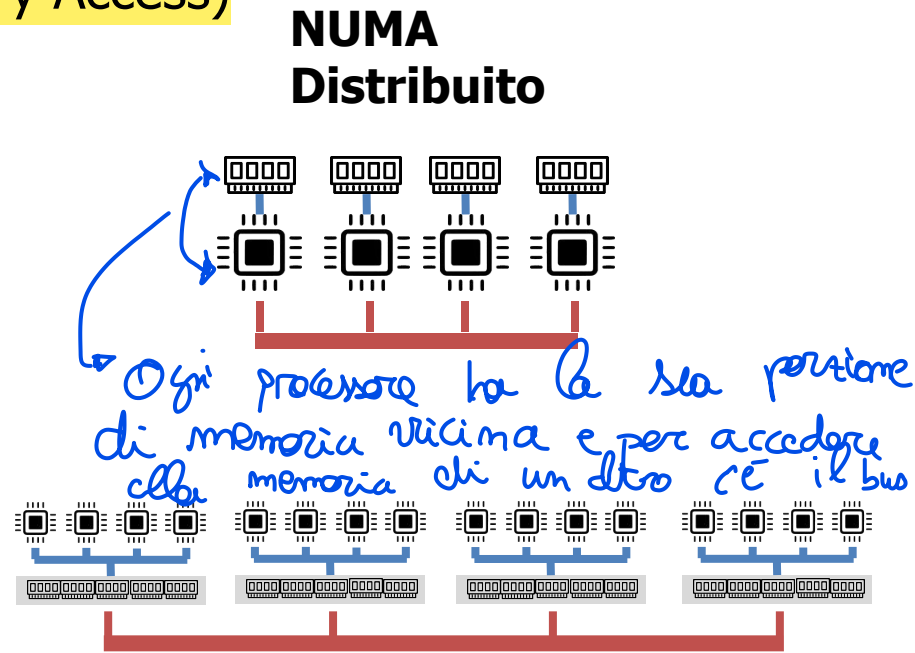
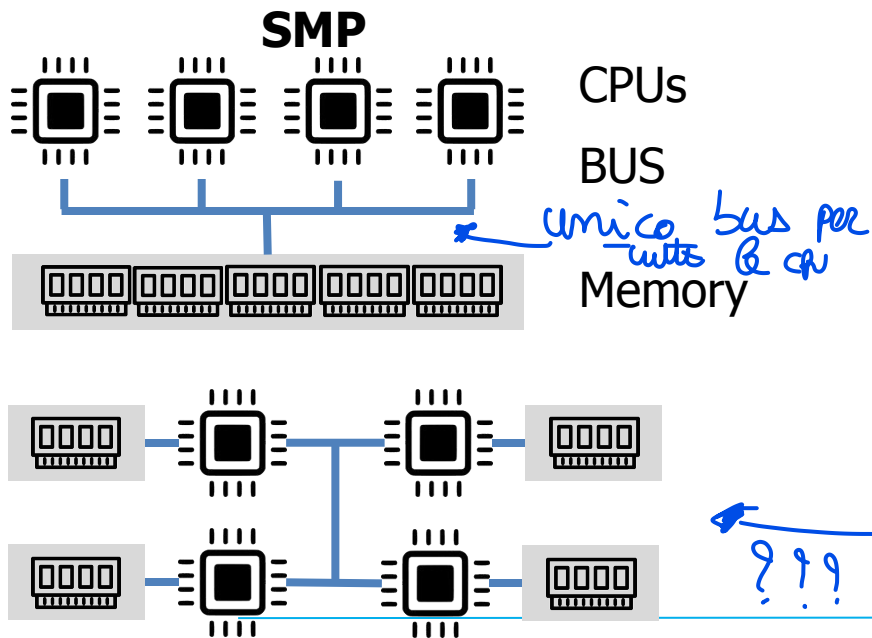
**Le 3R  
per  
gli amici**

- ▶ **Riorganizzare (codice e dati):** Progettare il layout per migliorare la località temporale e spaziale.
- ▶ **Ridurre (dimensione):** Dimensione dei dati più piccola, meno istruzioni.
- ▶ **Riutilizzare (linee di cache):** Aumentare la località per mantenere i dati residenti per più operazioni.



# Un problema di coerenza

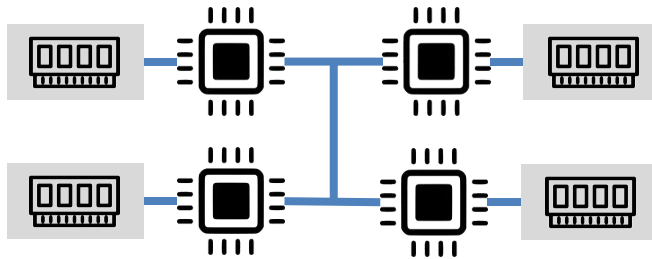
Conoscete già la differenza tra: **SMP (Symmetric Multiprocessing)** e **NUMA Distribuito (Non-Uniform Memory Access)**





# Un problema di coerenza

Considerate un nodo SMP, con decine di socket interconnesse da un bus.

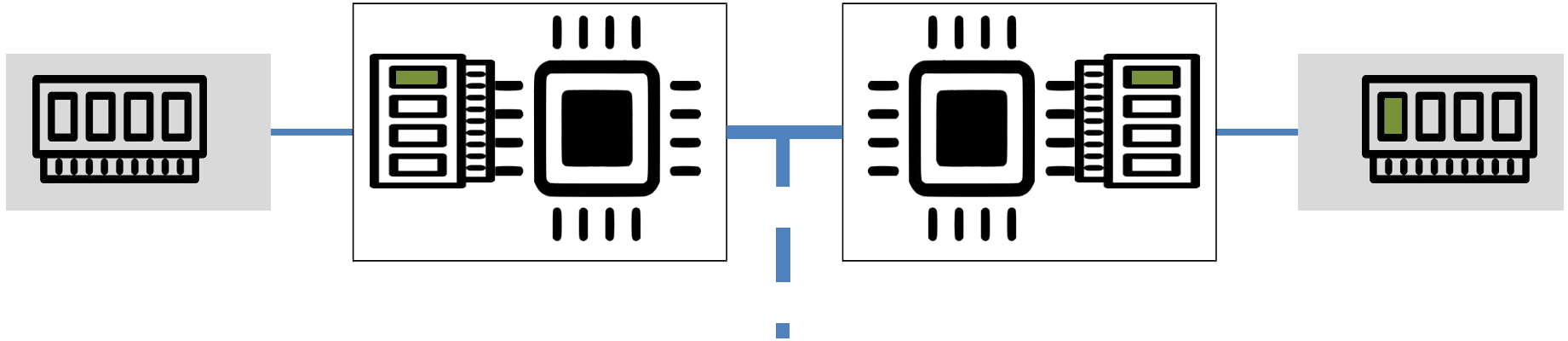


La memoria è *condivisa* e tutti vedono l'intera quantità di RAM.



# Un problema di coerenza

Supponiamo che le CPU abbiano cache e che alcuni dati siano caricati in più di una cache.



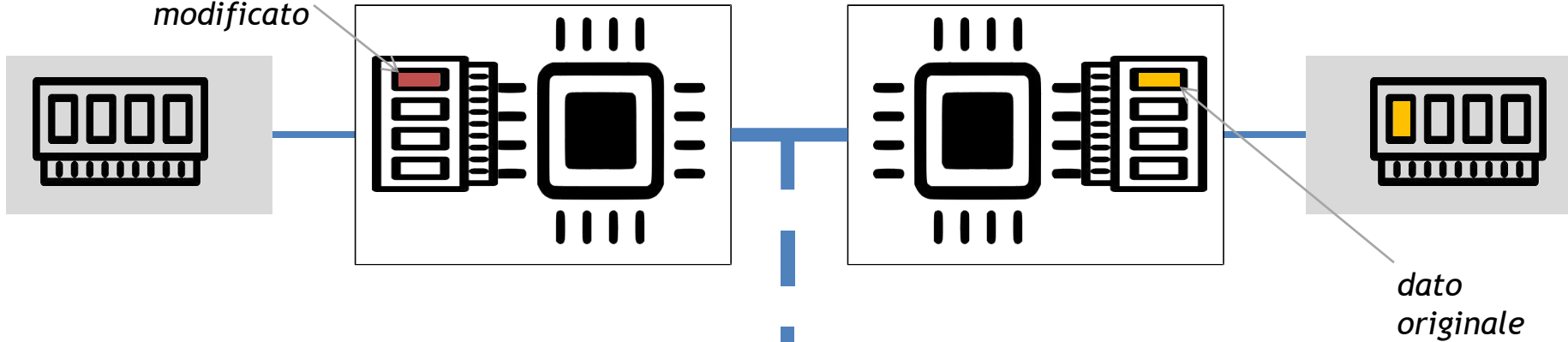


# Un problema di coerenza

Supponiamo che le CPU abbiano cache e che alcuni dati siano caricati in più di una cache.

*dato*

*modificato*



*dato  
originale*

Cosa succede a tutte le cache e ai dati "reali" in memoria quando una CPU modifica il dato?

Questo è chiamato *coerenza della cache* (cache coherency) e la difficoltà/costo per gestirla su nodi SMP troppo grandi è il principale limite alla loro dimensione.



# Inizio anni 90:

## le CPU diventano più veloci della memoria

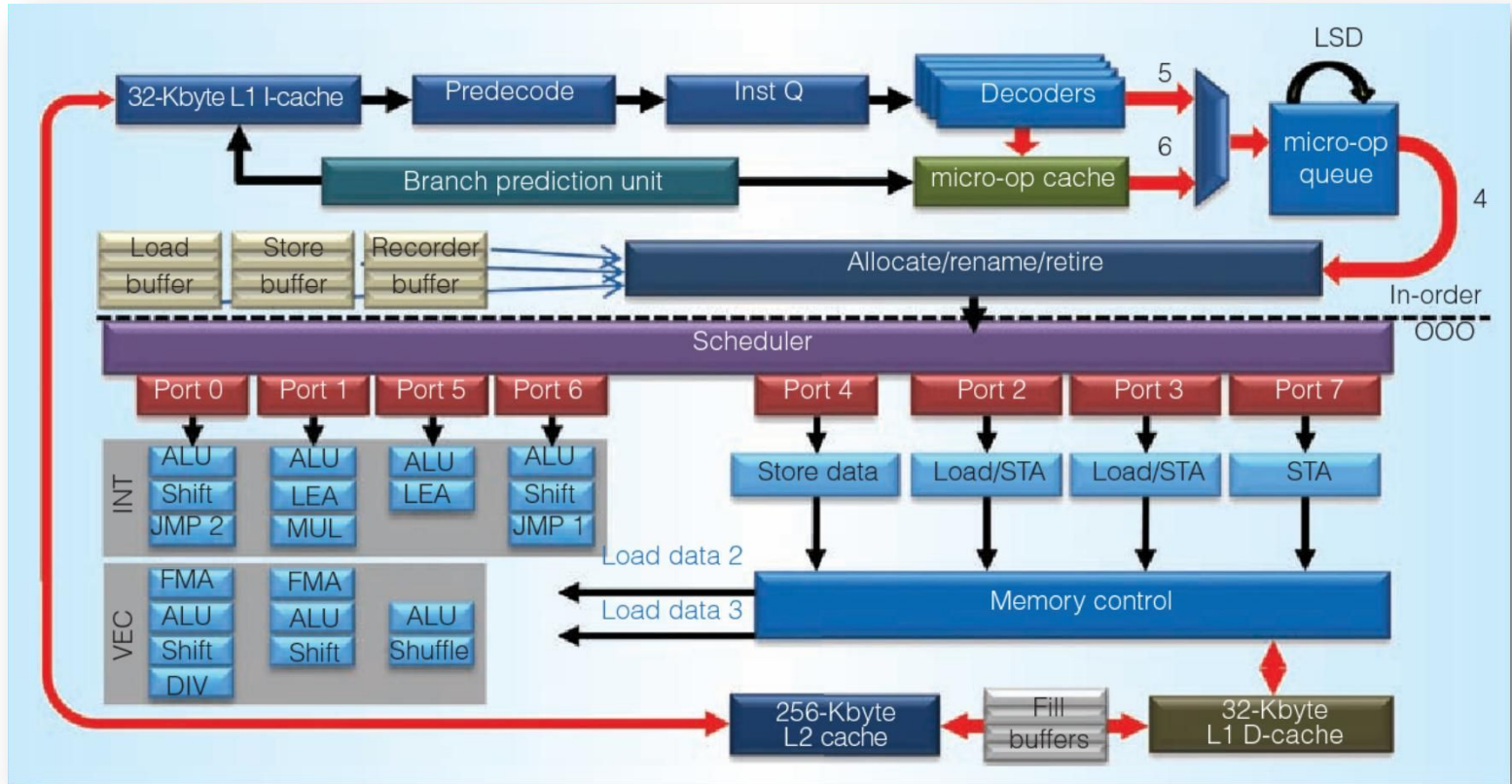
Dopo aver introdotto la gerarchia delle memorie cache, dovete affrontare una forte gerarchia di memoria e il fatto che la vostra CPU è molto più veloce della memoria centrale.

Questa complessità può portare a un disastro prestazionale (ad esempio, la CPU è bloccata in attesa di dati).

Questo ci porta ad ulteriori miglioramenti.



# Metà anni 90: CPU superscalari

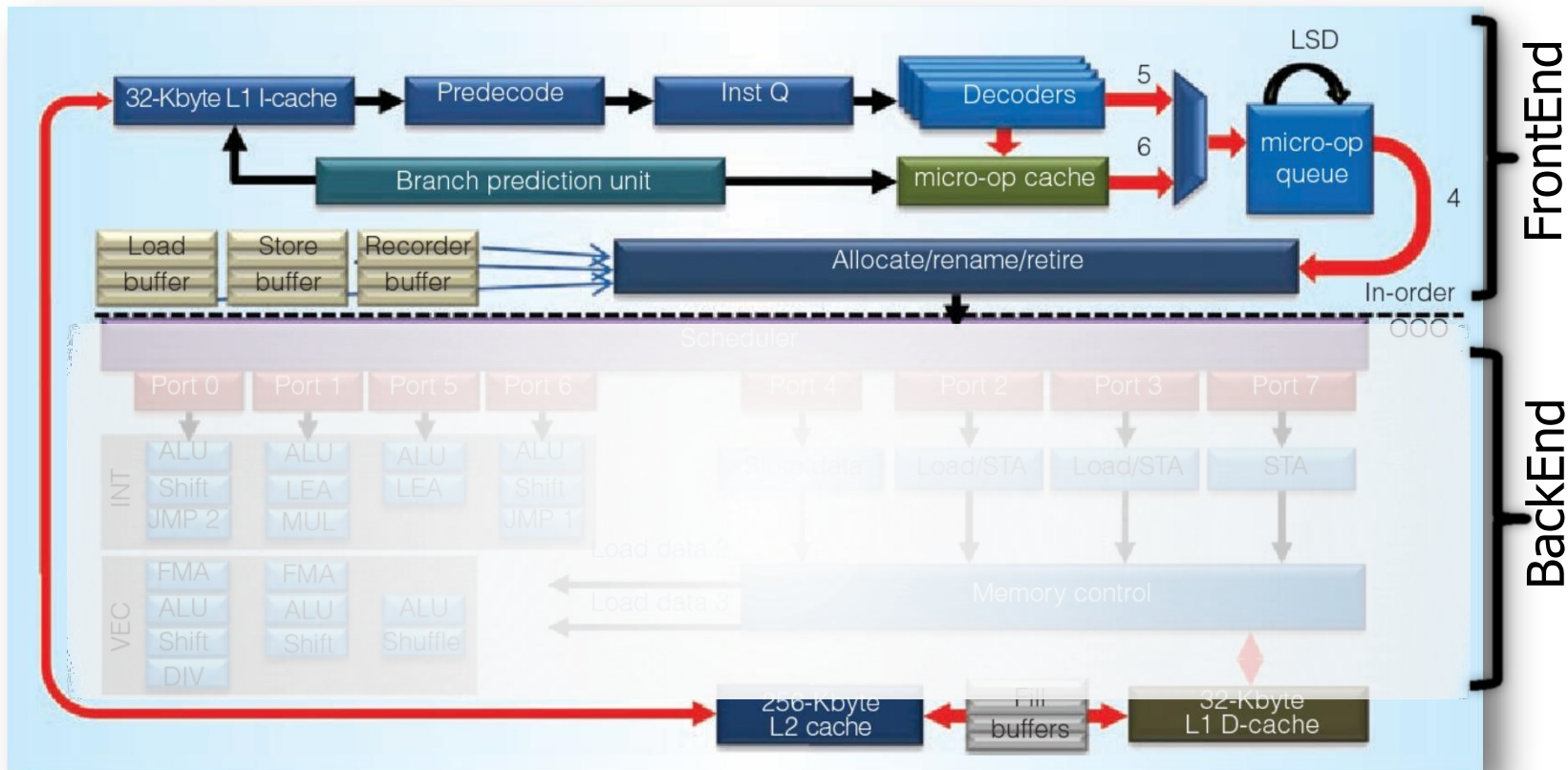


6<sup>th</sup> generation  
SkyLake  
micro-arch.



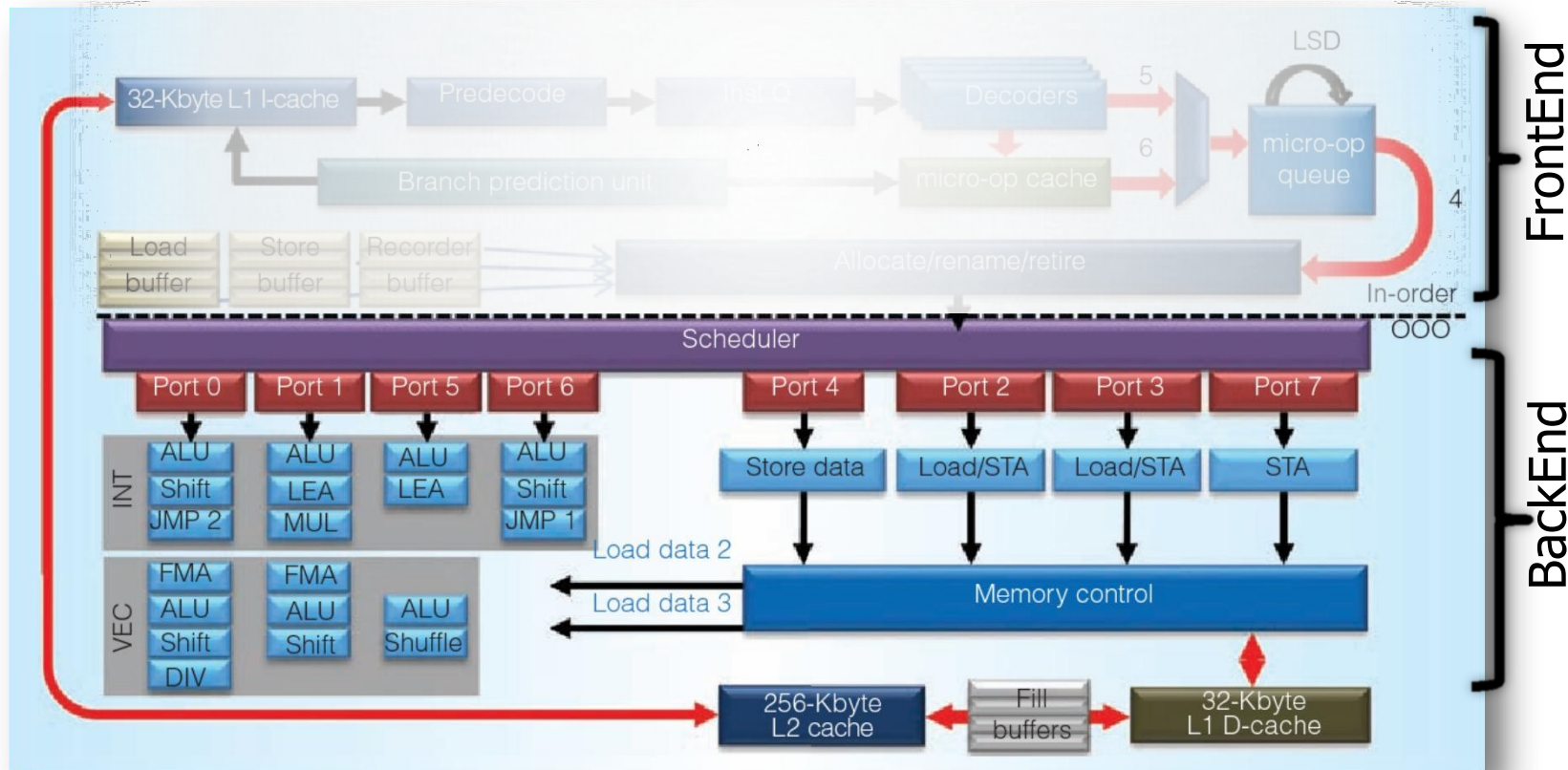


Il **Front-End** fondamentale recupera le istruzioni dalle cache, decodifica, predice i branch e smista le istruzioni alle diverse porte.



6<sup>th</sup> generation  
SkyLake  
micro-arch.

Il **Back-End** è responsabile dell'esecuzione effettiva delle istruzioni e della riscrittura dei risultati in memoria. È anche responsabile di orchestrare l'esecuzione delle operazioni out-of-order a seconda delle loro dipendenze.

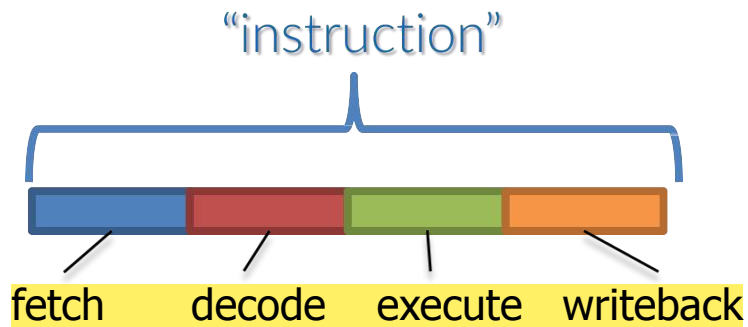




# Pipeline

Sarebbe ovvio pensare che una "istruzione" sia una sorta di operazione atomica che la CPU esegue nel suo complesso.

In effetti, questo era vero fino alla metà degli anni '80. Se ci pensate attentamente, è facile capire che in realtà un'"istruzione" comporta almeno i seguenti passaggi indipendenti:



## 1. **Fetch (Recupero):**

deve essere richiamata dalla memoria/Lcache.

## 2. **Decode (Decodifica):**

deve essere "compresa e interpretata".

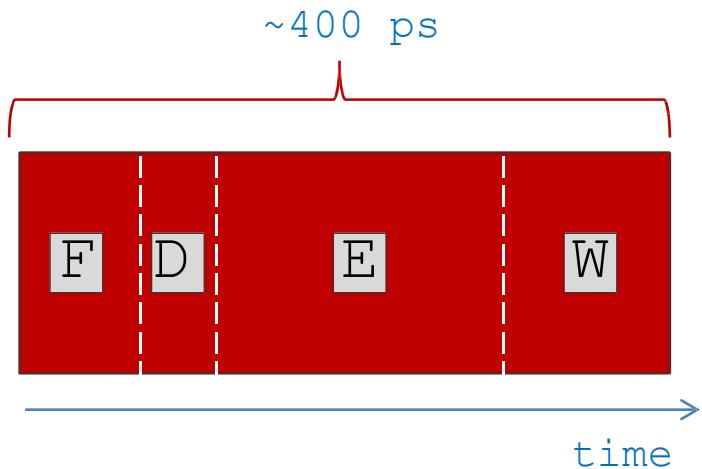
## 3. **Execute (Esecuzione)**

## 4. **Writeback (Scrittura del risultato):** il risultato deve essere registrato in memoria.



Pipeline

# Pipeline



Se tutti e quattro gli stadi richiedono  $\sim 400 \text{ ps}$ , si otterrebbe una produttività (**throughput**) di 2.5GIPS (giga-istruzioni al secondo).

400ps è anche la **latenza** dell'istruzione.

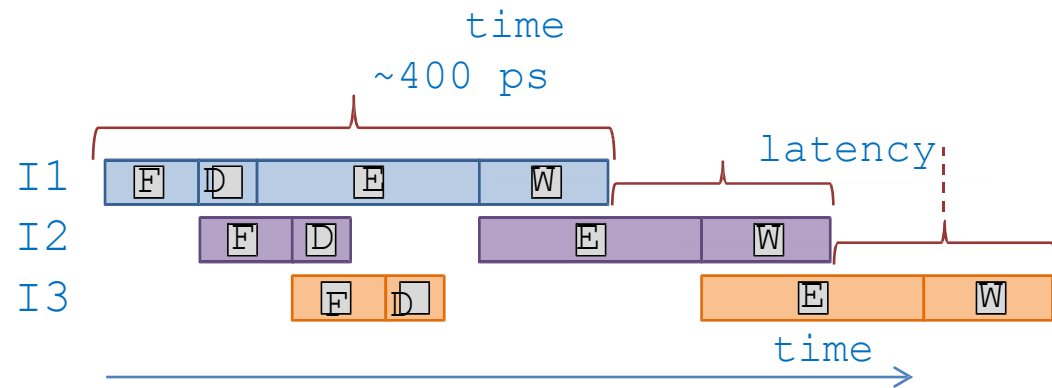
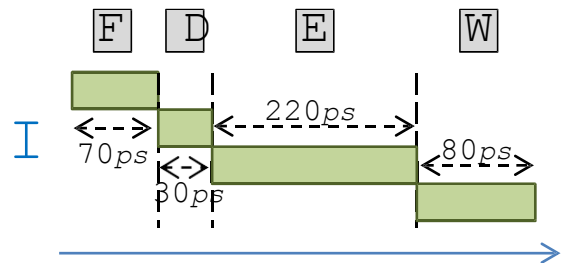
Se fossimo in grado di "scomporre" i quattro stadi, potremmo organizzare le cose diversamente (come in una catena di montaggio).



Pipeline

# Pipeline

*Nota: tutte le stime temporali sono ipotetiche ai fini della discussione dei concetti.*



Se esistono molte unità logiche indipendenti per eseguire ciascun passo, esse potrebbero operare in sequenza su **istruzioni diverse.**

La produttività/throughput è limitata dallo stadio più lungo (E ~ 220ps), il che significa una produttività di ~4.5GIPS solo grazie alla separazione delle unità logiche.

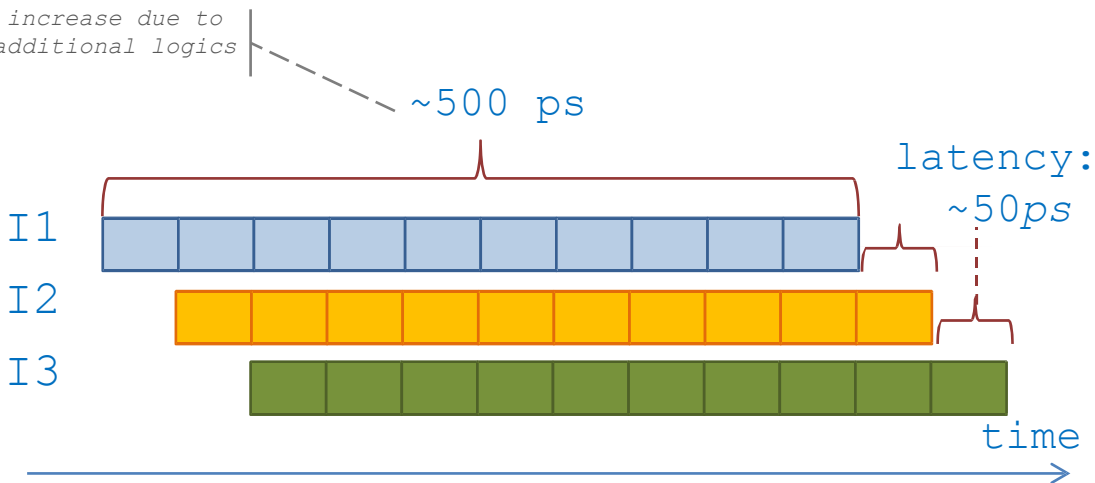


Pipeline

# Pipeline

Introducendo il **pipelining delle istruzioni**, possiamo aumentare la **produttività del sistema** (fino a 20GIPS con l'esempio). (\*).

*increase due to  
additional logics*



L'efficienza è limitata dallo **stadio più lungo**. L'opzione migliore è avere tutti stadi uguali.

(\*) questo processo è chiamato **superpipelining**: le CPU moderne possono avere da 10 a 20 fasi per pipeline.





# Pipeline e produttività

---

Il pipelining consiste quindi nell'aumentare la produttività di un sistema e, come abbiamo visto, può essere davvero efficace.

Tuttavia, si può fare di più lavorando sulla **fase di esecuzione** che, ovviamente, comprende un gran numero di diverse **unità funzionali**, che svolgono **compiti diversi e indipendenti**.

Con un miglioramento della fase di decodifica/invio, possiamo gestire più pipeline che possono essere attive contemporaneamente:

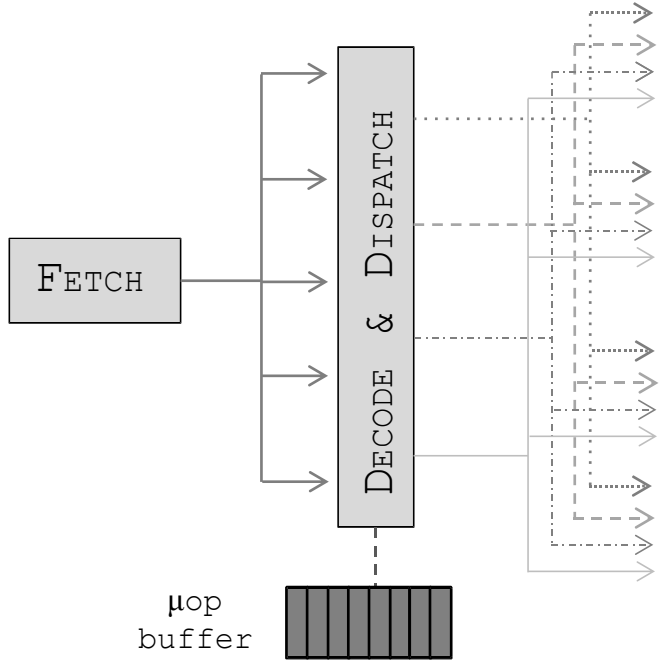
---





Pipeline

# Pipeline multiple





# I rischi delle Pipeline

---

Il pipelining è ottimale quando le istruzioni sono indipendenti ma la realtà è ben diversa...

**Data hazard:** un'istruzione dipende dal risultato di un'istruzione precedente che non ha ancora completato la pipeline.

**Control hazard:** quando la CPU incontra un branch condizionale, non sa quale ramo eseguirà finché non valuta la condizione, ma nel frattempo la pipeline ha già iniziato a fare il prefetch delle istruzioni successive.

---



# I rischi delle Pipeline

**Data hazard:** un'istruzione dipende dal risultato di un'istruzione precedente che non ha ancora completato la pipeline.

- ❖ Soluzione: La CPU deve inserire stalli (bubble) nella pipeline, perdendo cicli produttivi. → l'operazione è "messa in attesa" con

**Control hazard:** quando la CPU incontra un branch condizionale, non sa quale ramo eseguirà finché non valuta la condizione, ma nel frattempo la pipeline ha già iniziato a fare il prefetch delle istruzioni successive. *cicli nulli*

- ❖ Soluzione: branch predictor, un componente hardware che cerca di indovinare quale ramo verrà preso, basandosi sulla storia dei branch precedenti.

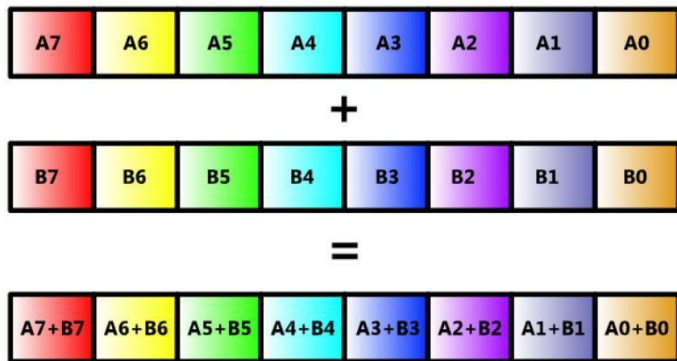
→ cerchiamo di indovinare il branch da eseguire



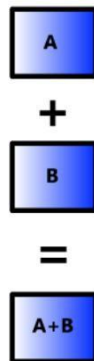
# Fine anni 90: Vettorizzazione

[https://software.intel.com/sites/default/files/m/d/4/1/d/8/Intro\\_to\\_Intel\\_AVX.pdf](https://software.intel.com/sites/default/files/m/d/4/1/d/8/Intro_to_Intel_AVX.pdf)

## SIMD Mode



## Scalar Mode



I **registri vettoriali** sono grandi registri speciali nella CPU, suddivisi in porzioni su cui può essere eseguita la stessa operazione:

SIMD: **Single Instruction Multiple Data**  
(Istruzione Singola, Dati Multipli).



# Dimensione dei registri vettoriali

## SSE Data Types (16 XMM Registers)

|         |                |       |        |       |                  |       |       |       |           |       |       |       |           |       |                  |                 |                 |
|---------|----------------|-------|--------|-------|------------------|-------|-------|-------|-----------|-------|-------|-------|-----------|-------|------------------|-----------------|-----------------|
| __m128  | Float          | Float | Float  | Float | 4x 32-bit float  |       |       |       |           |       |       |       |           |       |                  |                 |                 |
| __m128d | Double         |       | Double |       | 2x 64-bit double |       |       |       |           |       |       |       |           |       |                  |                 |                 |
| __m128i | B              | B     | B      | B     | B                | B     | B     | B     | B         | B     | B     | B     | B         | B     | B                | 16x 8-bit byte  |                 |
| __m128i | short          | short | short  | short | short            | short | short | short | short     | short | short | short | short     | short | short            | 8x 16-bit short |                 |
| __m128i | int            |       | int    |       | int              |       | int   |       | int       |       | int   |       | int       |       | 4x 32bit integer |                 |                 |
| __m128i | long long      |       |        |       | long long        |       |       |       | long long |       |       |       | long long |       |                  |                 | 2x 64bit long   |
| __m128i | doublequadword |       |        |       |                  |       |       |       |           |       |       |       |           |       |                  |                 | 1x 128-bit quad |

## AVX Data Types (16 YMM Registers)

|          |                                                                                                 |       |        |       |        |       |        |       |                  |
|----------|-------------------------------------------------------------------------------------------------|-------|--------|-------|--------|-------|--------|-------|------------------|
| __mm256  | Float                                                                                           | Float | Float  | Float | Float  | Float | Float  | Float | 8x 32-bit float  |
| __mm256d | Double                                                                                          |       | Double |       | Double |       | Double |       | 4x 64-bit double |
| __mm256i | 256-bit Integer registers. It behaves similarly to __m128i. Out of scope in AVX, useful on AVX2 |       |        |       |        |       |        |       |                  |



# Inizi 2000: multi-core

---

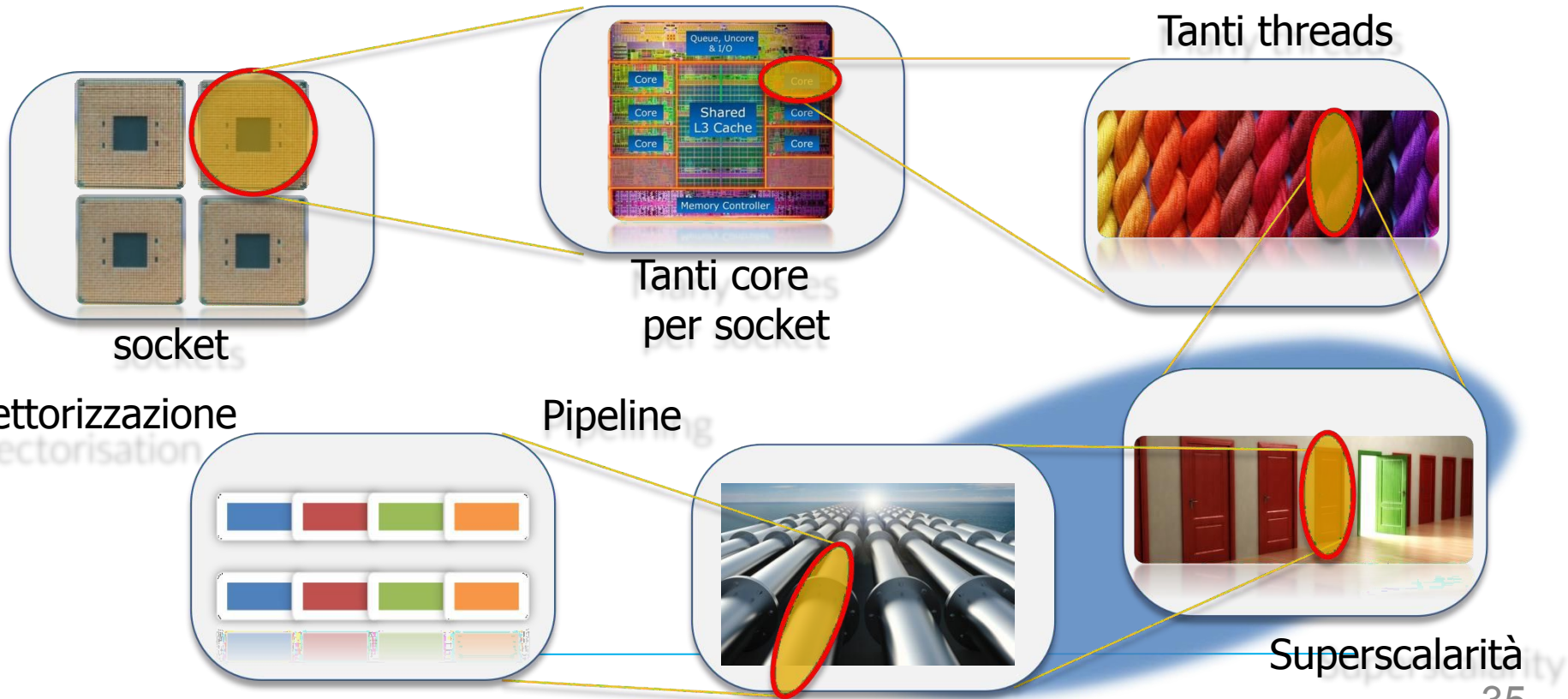
Fine anni '00: gli acceleratori diventano mainstream.

Inizio anni '10: il calcolo multi-core + eterogeneo diventa mainstream.

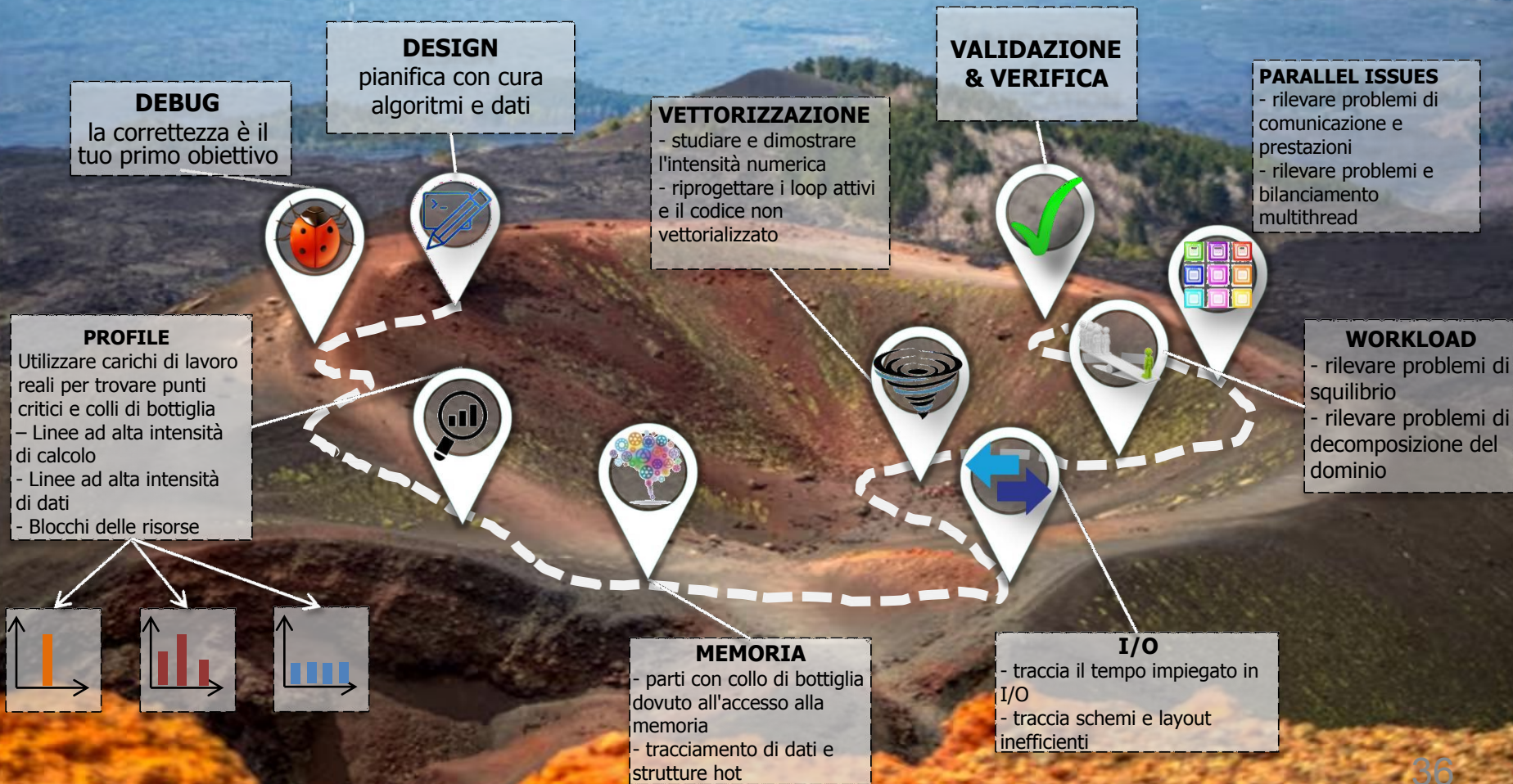
Questo è il focus del corso su OpenMP/MPI.



# Per riepilogare









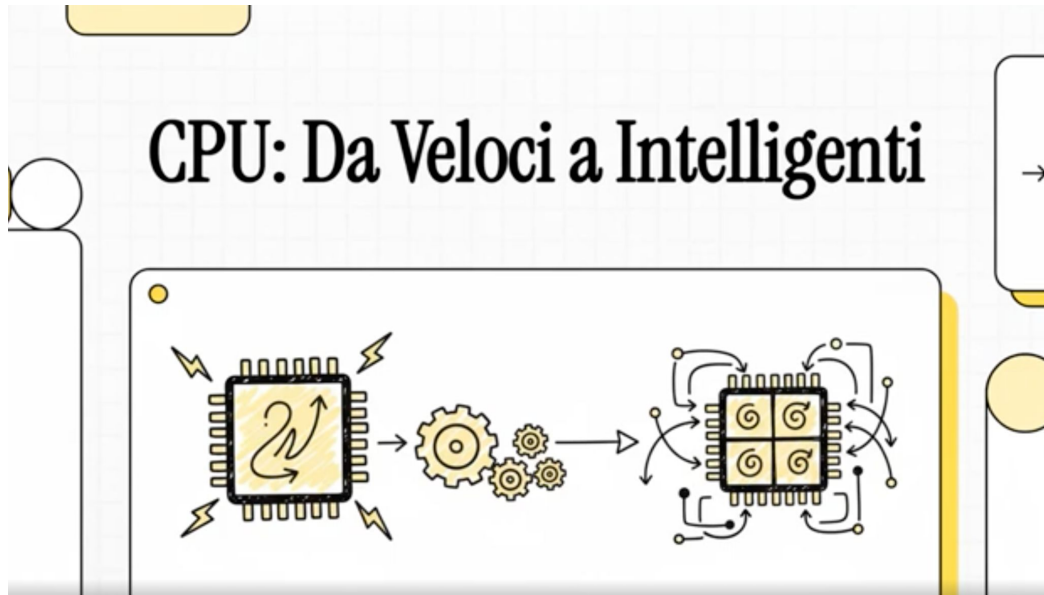


# Overview

Questo video è stato generato con strumenti di Intelligenza Artificiale (<https://notebooklm.google/>) utilizzando contenuti e fonti forniti dall'autore del corso.

Il materiale ha esclusivamente funzione di supporto e sintesi degli argomenti trattati a lezione. Non sostituisce il materiale didattico ufficiale, le slide, i testi di riferimento o lo studio individuale.

Il video deve essere inteso come uno strumento integrativo per il ripasso e il consolidamento dei concetti, e non come fonte esaustiva o alternativa allo studio approfondito.



[https://youtu.be/\\_gU9WepYpoE](https://youtu.be/_gU9WepYpoE)